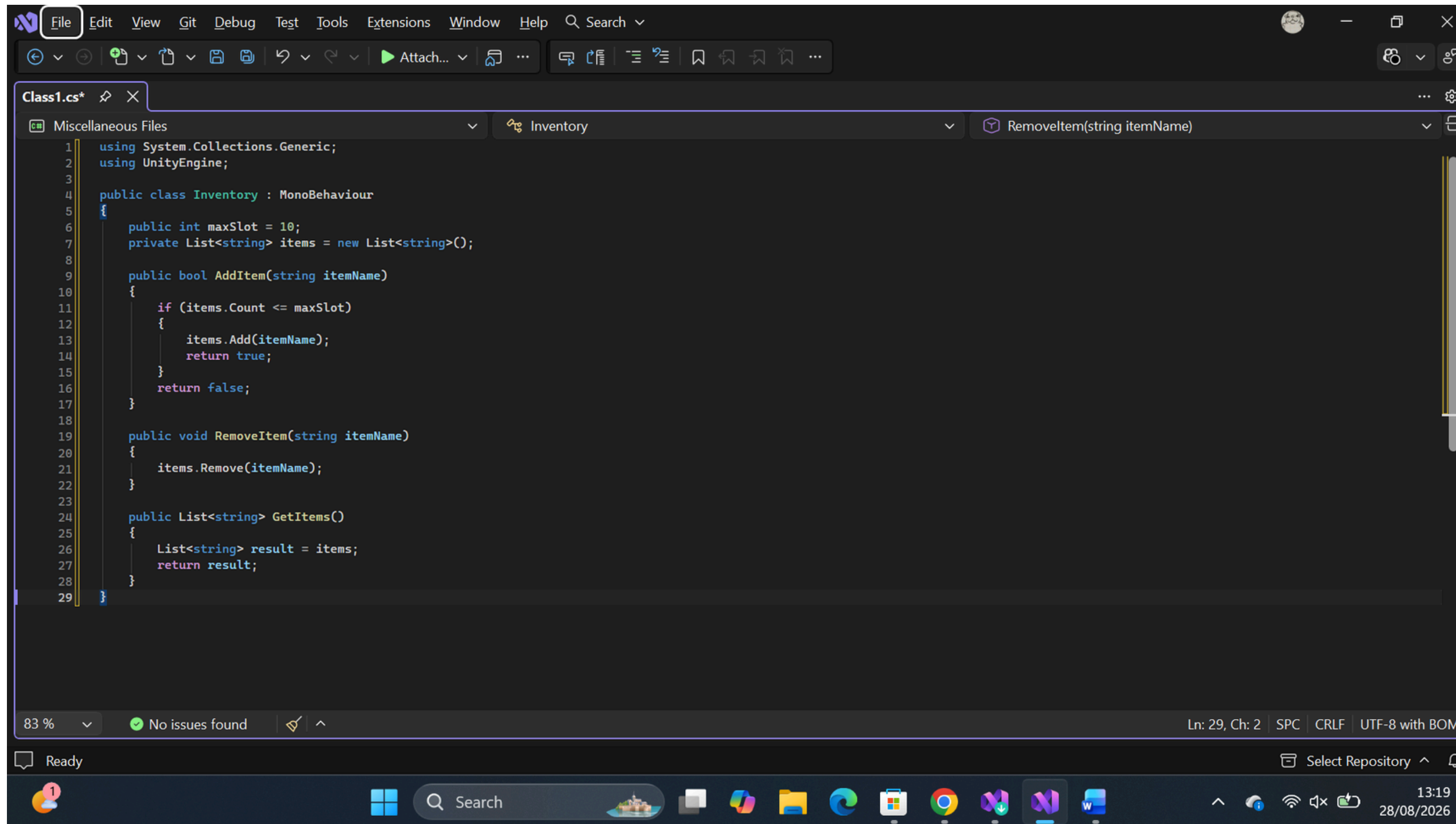


TANTANGAN 1 - DEBUGGING Kompleks: sistem Inventori (Level Sulit)



```
1  using System.Collections.Generic;
2  using UnityEngine;
3
4  public class Inventory : MonoBehaviour
5  {
6      public int maxSlot = 10;
7      private List<string> items = new List<string>();
8
9      public bool AddItem(string itemName)
10     {
11         if (items.Count <= maxSlot)
12         {
13             items.Add(itemName);
14             return true;
15         }
16         return false;
17     }
18
19     public void RemoveItem(string itemName)
20     {
21         items.Remove(itemName);
22     }
23
24     public List<string> GetItems()
25     {
26         List<string> result = items;
27         return result;
28     }
29 }
```

83 % No issues found Ln: 29, Ch: 2 SPC CRLF UTF-8 with BOM

Ready Select Repository

13:19 28/08/2026

Tugas

1. Identifikasi Kesalahan

- **Kesalahan 1: Baris 11 pada fungsi AddItem — if (items.Count <= maxSlot)**
- **Kesalahan 2: Baris 26–27 pada fungsi GetItems — List<string> result = items; return result;**

2. Penjelasan & Dampak

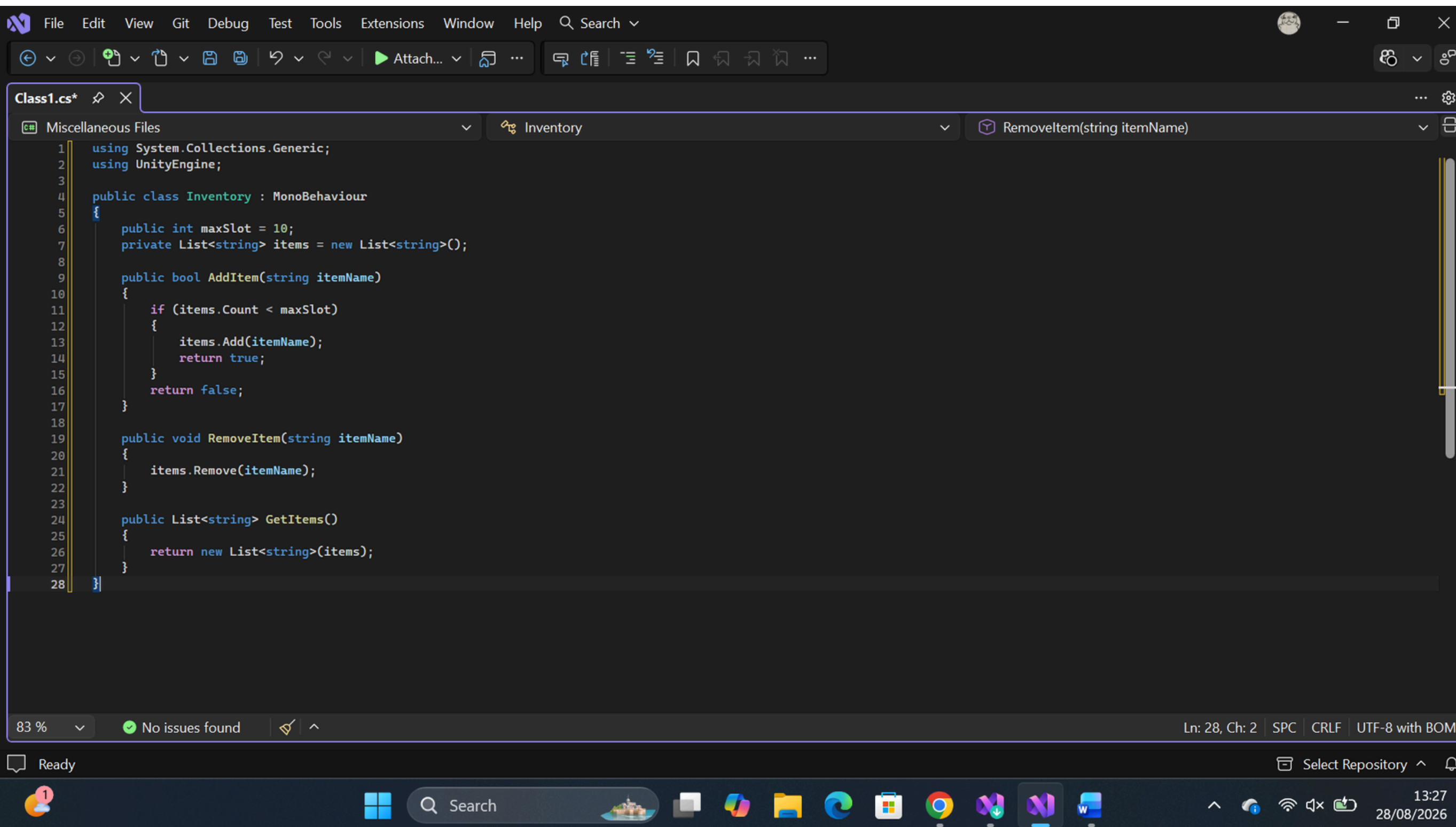
Baris 11:

- **Mengapa salah: Saat items.Count bernilai 10, kondisi $10 \leq 10$ bernilai true.**
- **Dampak: Inventaris bisa menampung 11 item (melebihi batas maxSlot = 10), merusak batas kapasitas UI/gameplay.**

Baris 26–27:

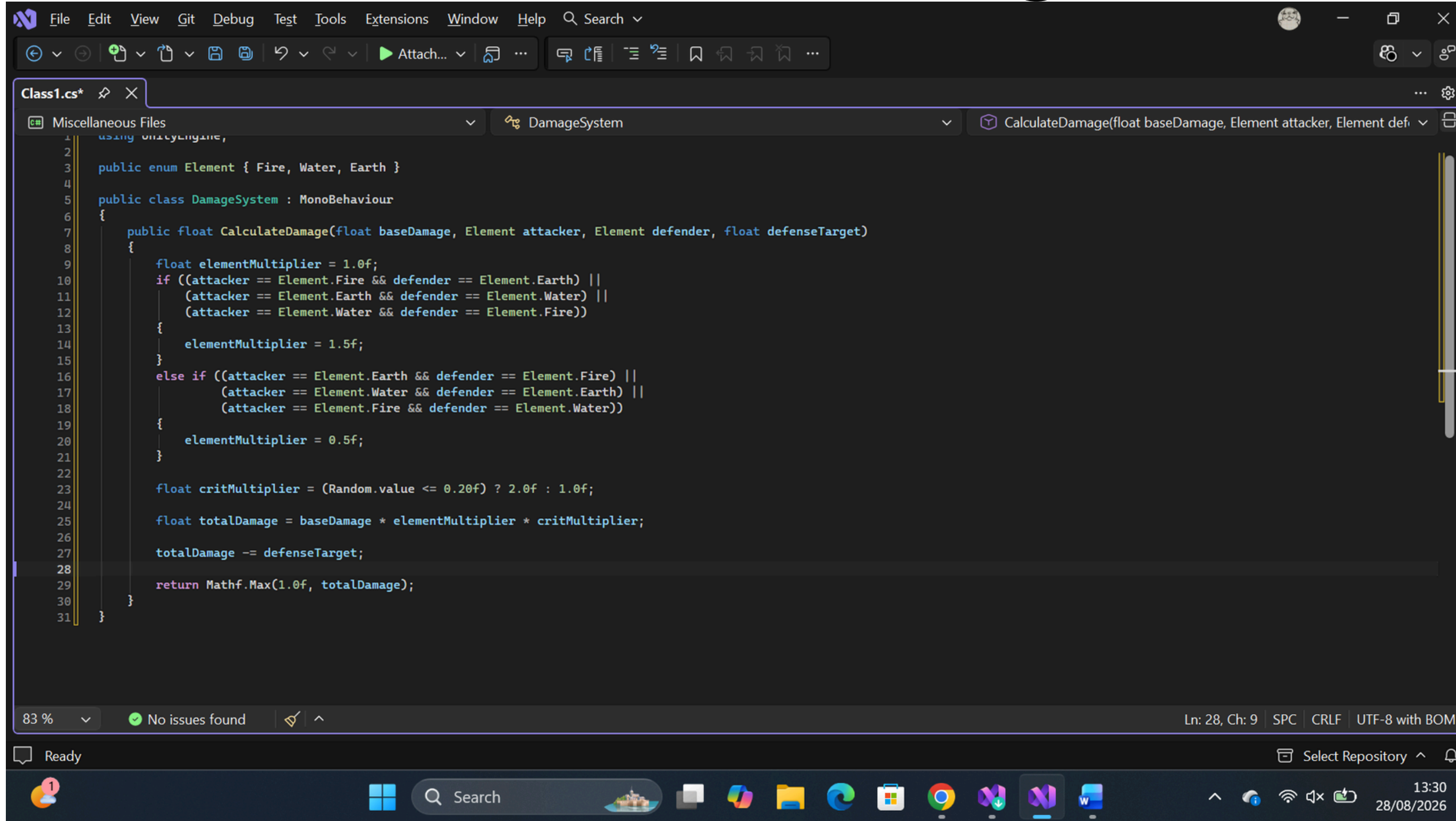
- **Mengapa salah: Mengembalikan referensi list asli ke luar kelas.**
- **Dampak: Kode luar bisa mengubah isi inventaris langsung (seperti memanggil `GetItems().Clear()`) tanpa melewati batas maxSlot atau fungsi resmi.**

3. Kode Perbaikan



TATANGAN 2

1. Kode C# (CalculateDamage)



The screenshot shows the Visual Studio Code editor with a C# file named `Class1.cs`. The code defines an enum `Element` with values `Fire`, `Water`, and `Earth`. It also defines a class `DamageSystem` that inherits from `MonoBehaviour`. The `CalculateDamage` method is implemented as follows:

```
1 using UnityEngine;
2
3 public enum Element { Fire, Water, Earth }
4
5 public class DamageSystem : MonoBehaviour
6 {
7     public float CalculateDamage(float baseDamage, Element attacker, Element defender, float defenseTarget)
8     {
9         float elementMultiplier = 1.0f;
10        if ((attacker == Element.Fire && defender == Element.Earth) ||
11            (attacker == Element.Earth && defender == Element.Water) ||
12            (attacker == Element.Water && defender == Element.Fire))
13        {
14            elementMultiplier = 1.5f;
15        }
16        else if ((attacker == Element.Earth && defender == Element.Fire) ||
17                (attacker == Element.Water && defender == Element.Earth) ||
18                (attacker == Element.Fire && defender == Element.Water))
19        {
20            elementMultiplier = 0.5f;
21        }
22
23        float critMultiplier = (Random.value <= 0.20f) ? 2.0f : 1.0f;
24
25        float totalDamage = baseDamage * elementMultiplier * critMultiplier;
26
27        totalDamage -= defenseTarget;
28
29        return Mathf.Max(1.0f, totalDamage);
30    }
31 }
```

The status bar at the bottom indicates 83% zoom, no issues found, and the file is encoded in UTF-8 with BOM. The system tray at the bottom shows the time as 13:30 on 28/08/2026.

2. Alasan Urutan Operasi

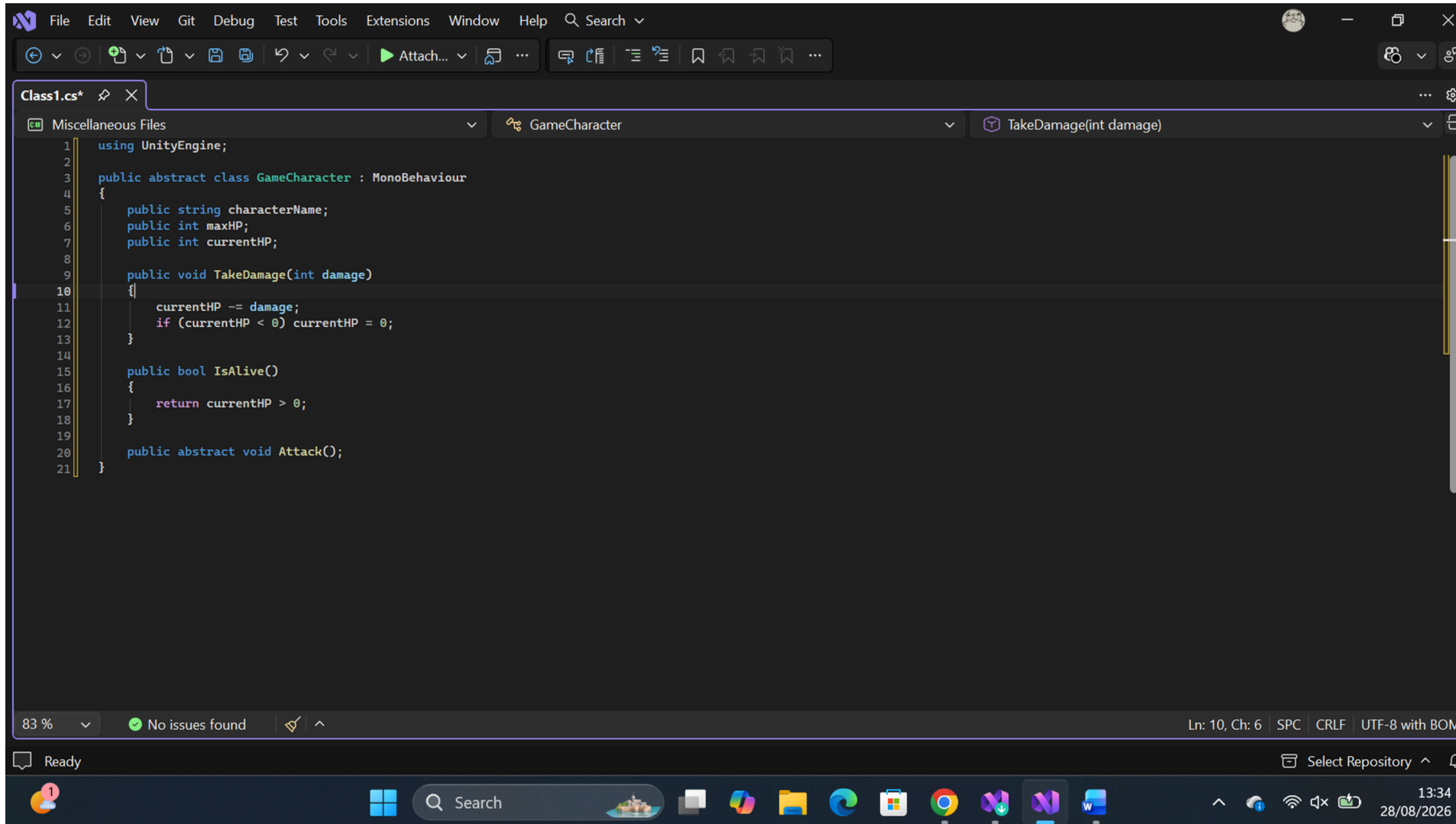
- Pengali (Elemen & Critical) Dihitung Pertama: Memperbesar nilai base damage terlebih dahulu sesuai mekanik efektivitas serangan dan keberuntungan critical.**
- Defense Pengurangan Flat Dihitung Setelah Pengali: Sesuai instruksi soal, pengurangan defense mengurangi hasil akhir dari total serangan. Jika defense dikurangi di awal, pengali elemen dan critical akan salah membesarkan nilai sisa damage.**
- Batas Minimal 1 Dihitung Terakhir: Mencegah hasil damage menjadi nol atau negatif jika nilai defense musuh sangat tinggi.**

3. Contoh Perhitungan Manual

- **Diketahui:**
 - **Damage Dasar = 40**
 - **Elemen Unggul = 1.5x**
 - **Critical Hit = 2x**
 - **Defense Target = 25**
- **Langkah 1 (Hitung Damage Pembesar): $40 \times 1.5 \times 2 = 120$**
- **Langkah 2 (Kurangi Defense): $120 - 25 = 95$**
- **Langkah 3 (Cek Minimal 1): Karena 95 lebih besar dari 1, maka hasil akhir tetap 95.**

TANTANGAN 3

1. Abstract class character

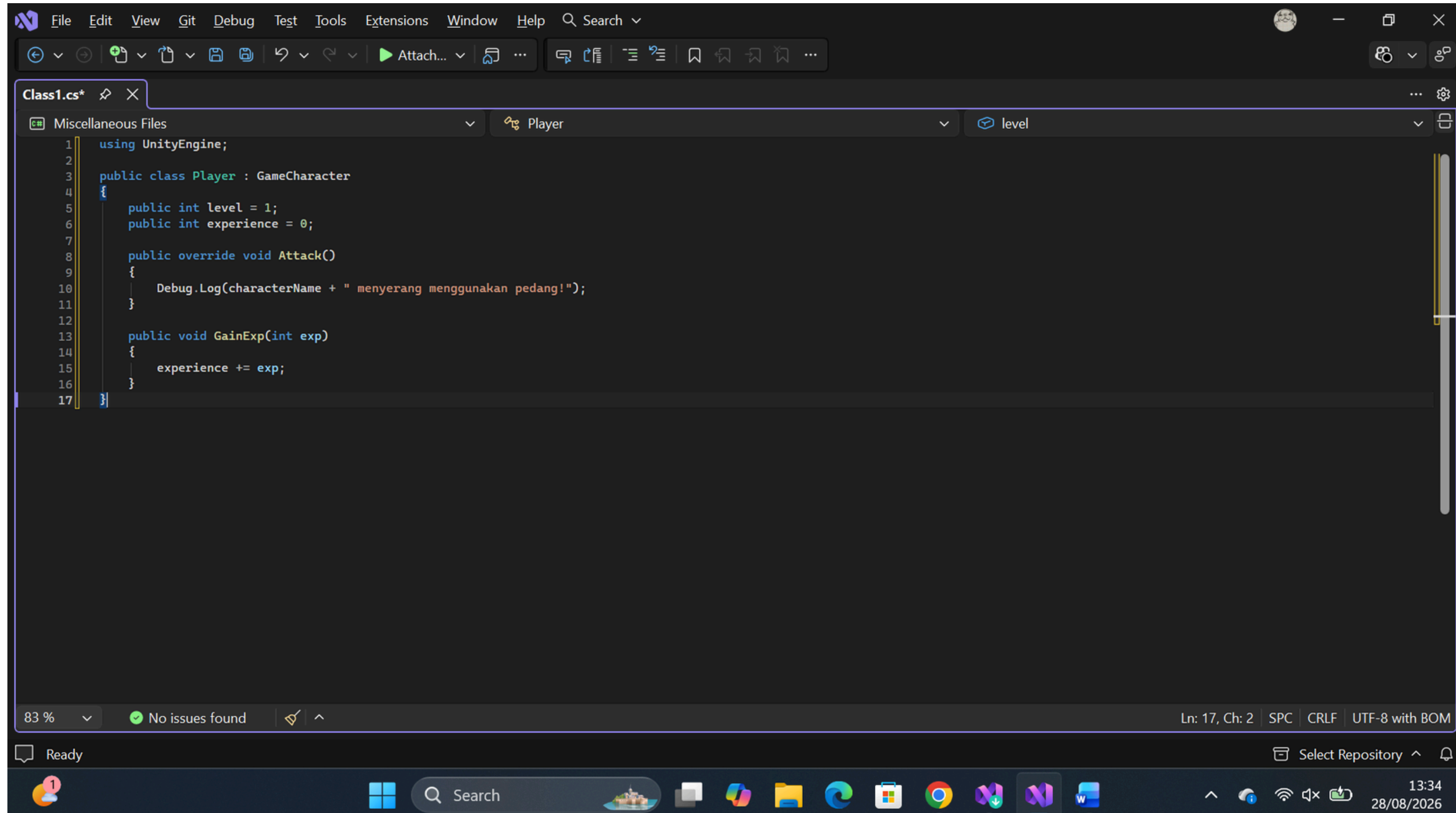


The screenshot shows the Visual Studio Code editor with a C# file named `Class1.cs`. The code defines an abstract class `GameCharacter` that inherits from `MonoBehaviour`. The class has three public fields: `characterName` (string), `maxHP` (int), and `currentHP` (int). It has two public methods: `TakeDamage(int damage)` and `IsAlive()`. The `TakeDamage` method decreases `currentHP` by the `damage` value and ensures it does not go below zero. The `IsAlive` method returns `true` if `currentHP` is greater than zero. There is also an abstract method `Attack()` that must be implemented by any class that inherits from `GameCharacter`.

```
1  using UnityEngine;
2
3  public abstract class GameCharacter : MonoBehaviour
4  {
5      public string characterName;
6      public int maxHP;
7      public int currentHP;
8
9      public void TakeDamage(int damage)
10     {
11         currentHP -= damage;
12         if (currentHP < 0) currentHP = 0;
13     }
14
15     public bool IsAlive()
16     {
17         return currentHP > 0;
18     }
19
20     public abstract void Attack();
21 }
```

The status bar at the bottom indicates that the file is 83% zoomed, there are no issues found, and the current line is 10, column 6. The encoding is UTF-8 with BOM.

2. Class Turunan (Player)



3. Mengapa Menggunakan Abstract Class, Bukan Interface?

- Abstract Class: Bisa menyimpan variabel (HP, Name) dan logika bersama (TakeDamage()), sehingga tidak perlu ditulis ulang.**
- Interface: Hanya berisi kontrak method, tidak bisa menyimpan variabel data.**

TANTANGAN 4

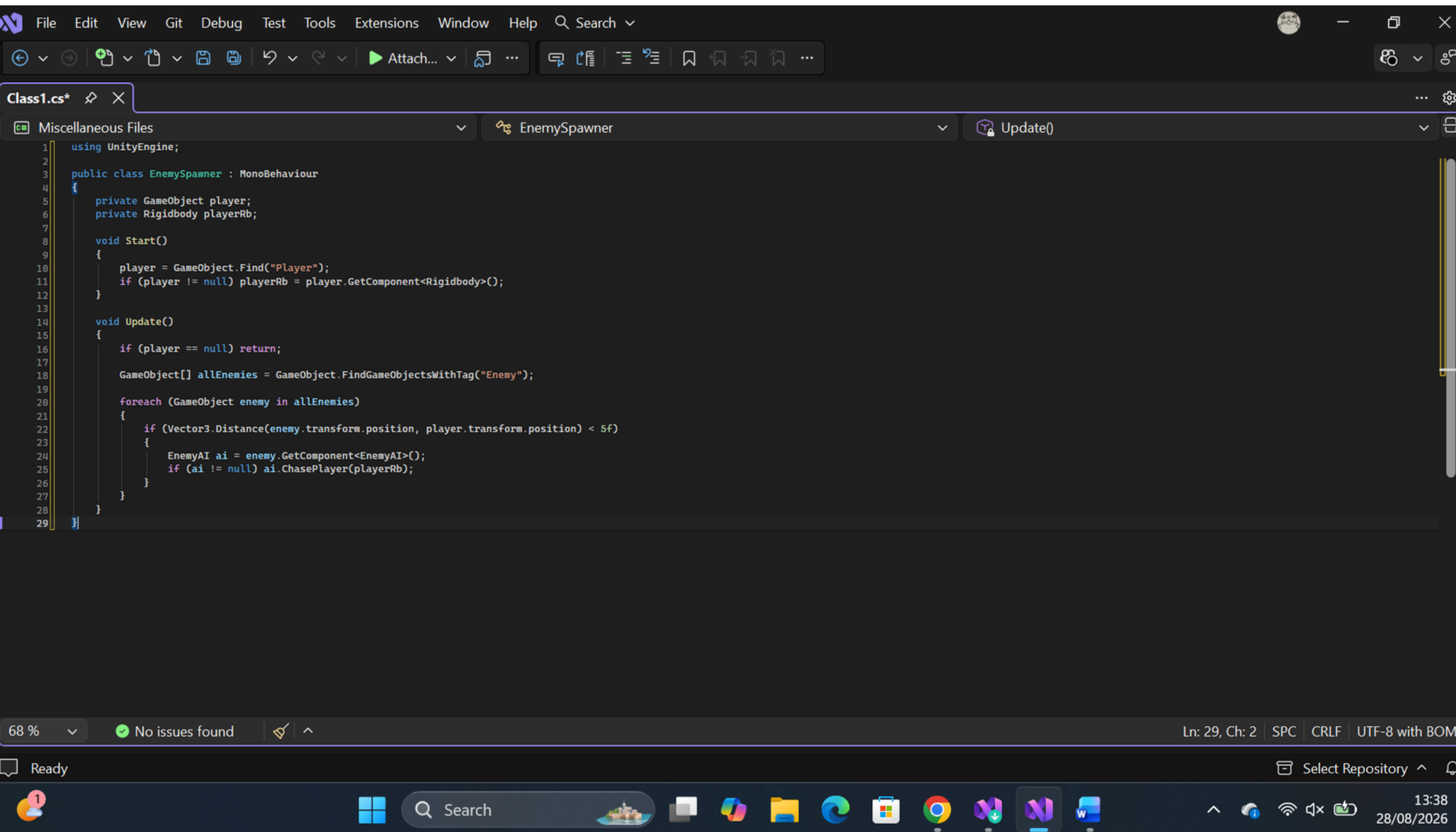
```
1 using UnityEngine;
2
3 public class EnemySpawner : MonoBehaviour
4 {
5     public GameObject[] allEnemiesInScene;
6
7     private GameObject player;
8     private Rigidbody playerRb;
9
10    void Start()
11    {
12        player = GameObject.Find("Player");
13        if (player != null)
14        {
15            playerRb = player.GetComponent<Rigidbody>();
16        }
17    }
18
19    void Update()
20    {
21        if (player == null) return;
22        allEnemiesInScene = GameObject.FindGameObjectsWithTag("Enemy");
23
24        foreach (GameObject enemy in allEnemiesInScene)
25        {
26            float distance = Vector3.Distance(enemy.transform.position, player.transform.position);
27
28            if (distance < 5f)
29            {
30                EnemyAI ai = enemy.GetComponent<EnemyAI>();
31                if (ai != null)
32                {
33                    ai.ChasePlayer(playerRb);
34                }
35            }
36        }
37    }
38 }
```

68 % No issues found Ln: 38, Ch: 2 SPC CRLF UTF-8 with BOM

1. Masalah Performa

- GameObject.Find("Player") di Update(): Mencari objek ke seluruh hierarchy 60x/detik membuat CPU sangat berat.**
- GetComponent di Update(): Mencari komponen berulang-ulang setiap frame memboroskan pemrosesan CPU.**
- FindGameObjectsWithTag("Enemy") di Update(): Membuat array baru terus-menerus, memicu Garbage Collector yang membuat game patah-patah (stuttering).**

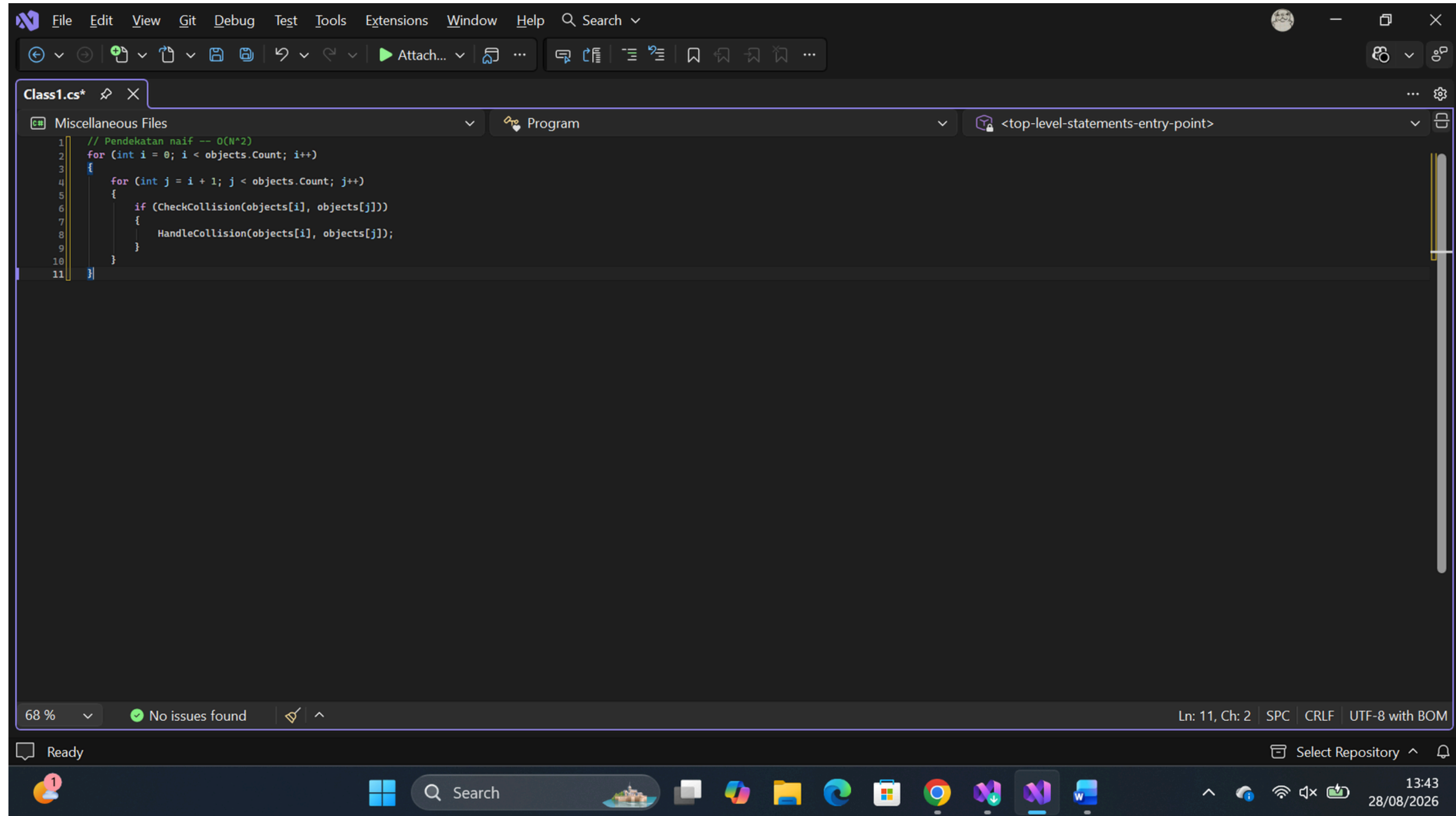
2. Kode Optimal



3. Penjelasan Soal No. 3

Tidak selalu salah. Dipanggil di Update() dapat dimaklumi jika musuh sering muncul (spawn) dan mati secara dinamis. Namun, cara yang lebih baik adalah musuh mendaftarkan dirinya ke sebuah daftar (list) saat baru muncul, daripada dicari ulang setiap frame.

TANTANGAN 5



TUGAS

1. Pembuktian Kombinatorika & Kompleksitas

- **Pembuktian:** Memilih 2 objek dari N objek tanpa memedulikan urutan menggunakan kombinasi:

$$\binom{N}{2} = \frac{N(N-1)}{2}$$

- **Kompleksitas $O(N^2)$:** Nilai $\frac{N(N-1)}{2} = \frac{1}{2}N^2 - \frac{1}{2}N$. Pada Big-O, suku dengan pangkat tertinggi yang diambil, sehingga menjadi $O(N^2)$.
- **Perhitungan $N = 1000$:**

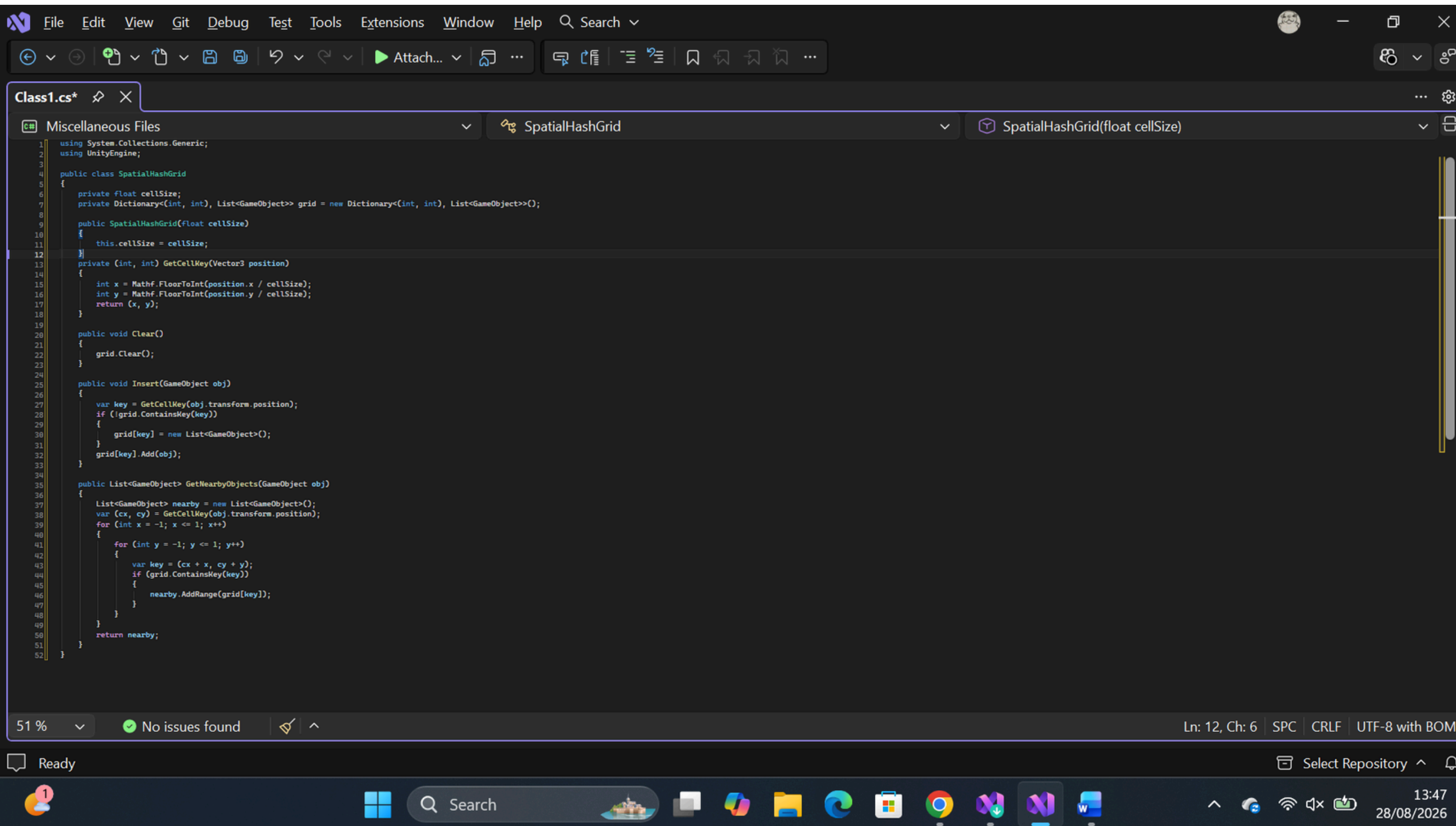
$$\frac{1000 \times 999}{2} = 499.500 \text{ kali per frame}$$

- **Mengapa Tidak Layak pada 60 FPS:** Pada 60 FPS, waktu per *frame* hanya 16,6 milidetik. Memproses ~500 ribu pengecekan tiap *frame* terlalu berat untuk CPU dan pasti menyebabkan *lag* parah.

2. Solusi & Kompleksitas Spatial Hash Grid

- **Cara Kerja:** Layar dibagi menjadi kotak-kotak (grid). Setiap objek dimasukkan ke koordinat sel tempatnya berada. Pengecekan tabrakan hanya dilakukan antar-objek yang berada di sel yang sama atau sel di sekitarnya.
- **Penurunan Kompleksitas $O(N)$:** Jika N objek tersebar merata ke dalam K sel, tiap sel berisi rata-rata $\frac{N}{K}$ objek. Pengecekan per sel menjadi $O(1)$, sehingga total pengecekan untuk seluruh objek mendekati $O(N)$.

3. Kode C# (SpatialHashGrid)



The screenshot shows the Visual Studio IDE with a C# file named `Class1.cs` open. The code implements a `SpatialHashGrid` class. The file explorer on the left shows the project structure: `Miscellaneous Files` > `SpatialHashGrid` > `SpatialHashGrid(float cellSize)`. The code is as follows:

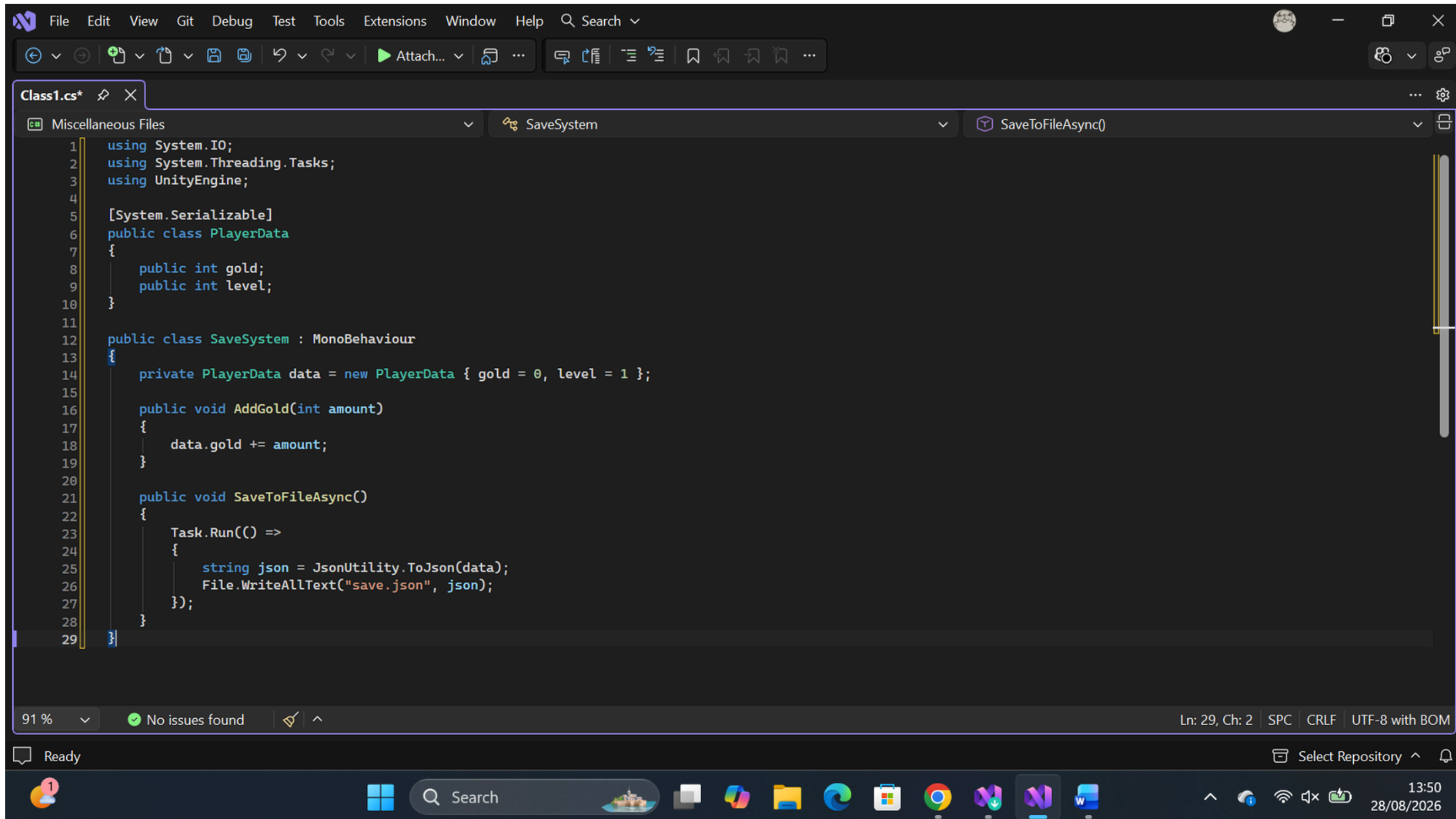
```
1 using System.Collections.Generic;
2 using UnityEngine;
3
4 public class SpatialHashGrid
5 {
6     private float cellSize;
7     private Dictionary<int, int>, List<GameObject>>> grid = new Dictionary<int, int>, List<GameObject>>>();
8
9     public SpatialHashGrid(float cellSize)
10    {
11        this.cellSize = cellSize;
12    }
13    private (int, int) GetCellKey(Vector3 position)
14    {
15        int x = Mathf.FloorToInt(position.x / cellSize);
16        int y = Mathf.FloorToInt(position.y / cellSize);
17        return (x, y);
18    }
19
20    public void Clear()
21    {
22        grid.Clear();
23    }
24
25    public void Insert(GameObject obj)
26    {
27        var key = GetCellKey(obj.transform.position);
28        if (!grid.ContainsKey(key))
29        {
30            grid[key] = new List<GameObject>();
31        }
32        grid[key].Add(obj);
33    }
34
35    public List<GameObject> GetNearbyObjects(GameObject obj)
36    {
37        List<GameObject> nearby = new List<GameObject>();
38        var (cx, cy) = GetCellKey(obj.transform.position);
39        for (int x = -1; x <= 1; x++)
40        {
41            for (int y = -1; y <= 1; y++)
42            {
43                var key = (cx + x, cy + y);
44                if (grid.ContainsKey(key))
45                {
46                    nearby.AddRange(grid[key]);
47                }
48            }
49        }
50        return nearby;
51    }
52 }
```

The status bar at the bottom indicates 51% zoom, no issues found, and the file is encoded in UTF-8 with BOM. The Windows taskbar at the very bottom shows the time as 13:47 on 28/08/2026.

4. Diskusi Kritis

- (a) Worst-Case: Jika semua objek berkumpul di 1 sel yang sama, semua objek tetap saling diperiksa satu sama lain, sehingga kompleksitasnya kembali memburuk menjadi $O(N^2)$.
- (b) Trade-Off Ukuran Sel (Cell Size):
 - Terlalu Kecil: Objek cepat berpindah sel dan boros memori karena membuat terlalu banyak sel.
 - Terlalu Besar: Banyak objek menumpuk di satu sel, sehingga deteksi kembali lambat seperti pendekatan naif.
- (c) Keunggulan Quadtree: Quadtree lebih unggul jika distribusi objek tidak merata (ada area sangat padat dan area sangat kosong). Quadtree dapat membagi area padat menjadi bagian-bagian yang lebih kecil secara adaptif.

TANTANGAN 6



```
1  using System.IO;
2  using System.Threading.Tasks;
3  using UnityEngine;
4
5  [System.Serializable]
6  public class PlayerData
7  {
8      public int gold;
9      public int level;
10 }
11
12 public class SaveSystem : MonoBehaviour
13 {
14     private PlayerData data = new PlayerData { gold = 0, level = 1 };
15
16     public void AddGold(int amount)
17     {
18         data.gold += amount;
19     }
20
21     public void SaveToFileAsync()
22     {
23         Task.Run(() =>
24         {
25             string json = JsonUtility.ToJson(data);
26             File.WriteAllText("save.json", json);
27         });
28     }
29 }
```

91 % No issues found Ln: 29, Ch: 2 SPC CRLF UTF-8 with BOM

1. Identifikasi Race Condition & Skenario Interleaving

- **Masalah:** Objek data diakses dan diubah oleh Main Thread (via **AddGold**), sementara Background Thread (**Task.Run**) membaca objek data yang sama untuk diubah menjadi JSON.
- **Skenario Interleaving:**
 - 1. Main Thread memanggil `SaveToFileAsync()`, memulai Background Thread.**
 - 2. Background Thread mulai membaca bidang `data.gold` untuk serialisasi JSON.**
 - 3. Di saat bersamaan, Main Thread memanggil `AddGold(50)` dan mengubah `data.gold`.**
 - 4. Background Thread lanjut membaca `data.level`.**
- **Hasil:** JSON menyimpan data setengah baru dan setengah lama (data tidak konsisten/korup).

2. Batasan API Unity pada Background Thread

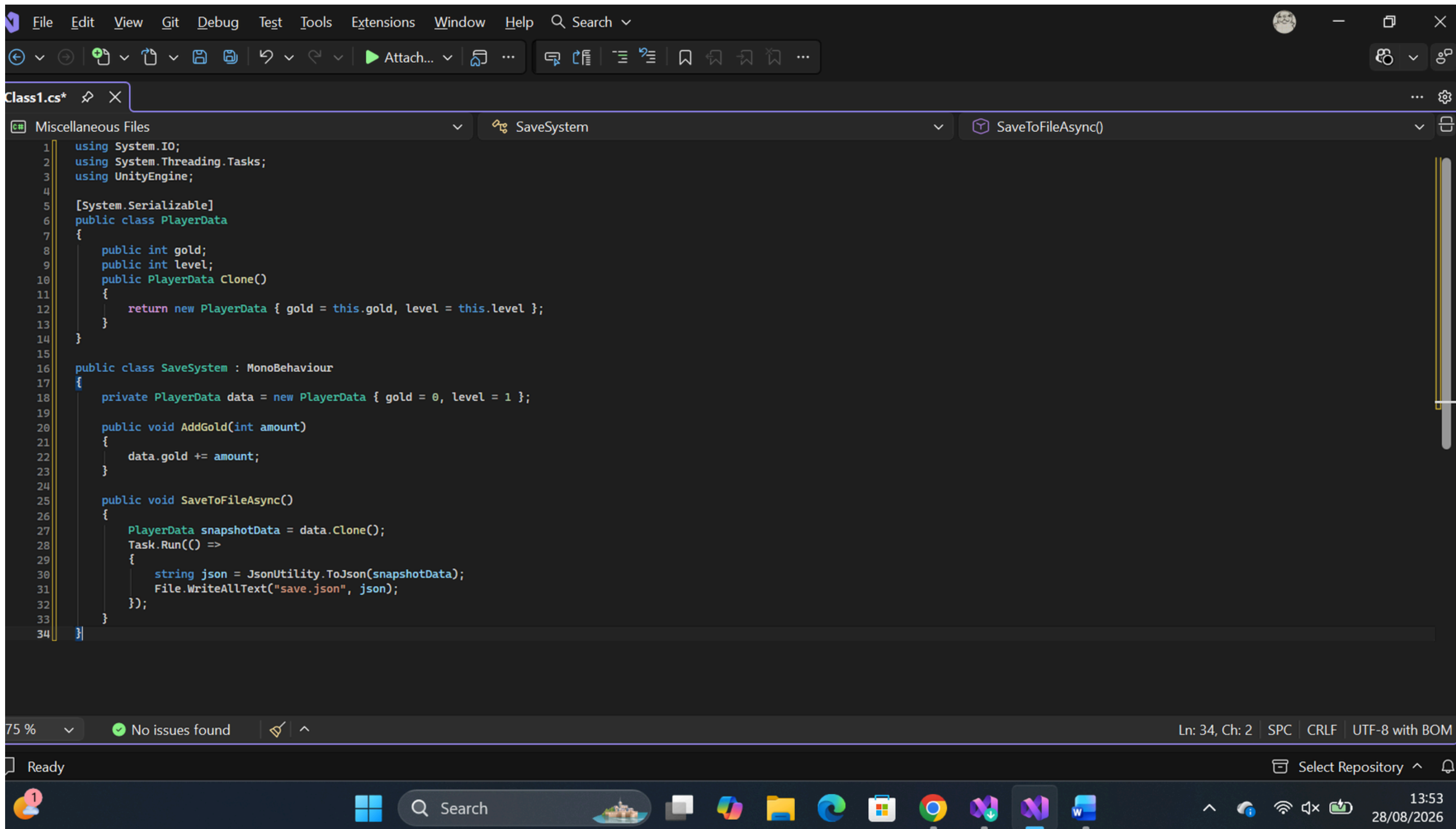
- Mengapa Error/Crash: Unity menggunakan arsitektur Single-Threaded Main Loop dan Thread Affinity.**

Sebagian besar API Unity (seperti Transform, GameObject, UnityEngine.Object) tidak thread-safe dan terikat secara ketat hanya pada Main Thread.

- Akibat: Memanggil API Unity di luar Main Thread akan memicu UnityException:**

Internal_CreateGameObject can only be called from the main thread untuk mencegah korupsi memori pada internal C++ Unity Engine.

3. Rancangan Kode Thread-Safe (Pendekatan B: Snapshot Data)



```
1 using System.IO;
2 using System.Threading.Tasks;
3 using UnityEngine;
4
5 [System.Serializable]
6 public class PlayerData
7 {
8     public int gold;
9     public int level;
10    public PlayerData Clone()
11    {
12        return new PlayerData { gold = this.gold, level = this.level };
13    }
14 }
15
16 public class SaveSystem : MonoBehaviour
17 {
18     private PlayerData data = new PlayerData { gold = 0, level = 1 };
19
20     public void AddGold(int amount)
21     {
22         data.gold += amount;
23     }
24
25     public void SaveToFileAsync()
26     {
27         PlayerData snapshotData = data.Clone();
28         Task.Run(() =>
29         {
30             string json = JsonUtility.ToJson(snapshotData);
31             File.WriteAllText("save.json", json);
32         });
33     }
34 }
```

4. Risiko Deadlock & Keunggulan Pendekatan Snapshot

- **Risiko Deadlock:** Penguncian (lock) berisiko menyebabkan deadlock jika dua thread saling menunggu kunci yang dipegang satu sama lain (misal: Thread A mengunci Resource 1 lalu menunggu Resource 2, sedangkan Thread B mengunci Resource 2 lalu menunggu Resource 1). Game akan membeku (freeze) total.
- **Mengapa Snapshot Lebih Disukai:**
- **Bebas Lock:** Mengeliminasi penggunaan lock, sehingga 100% aman dari risiko deadlock.
- **Performa Lebih Stabil:** Main Thread tidak perlu menunggu (blocking) operasi I/O file yang lambat, dan Background Thread bekerja pada salinan datanya sendiri tanpa mengganggu eksekusi game.

TANTANGAN 7

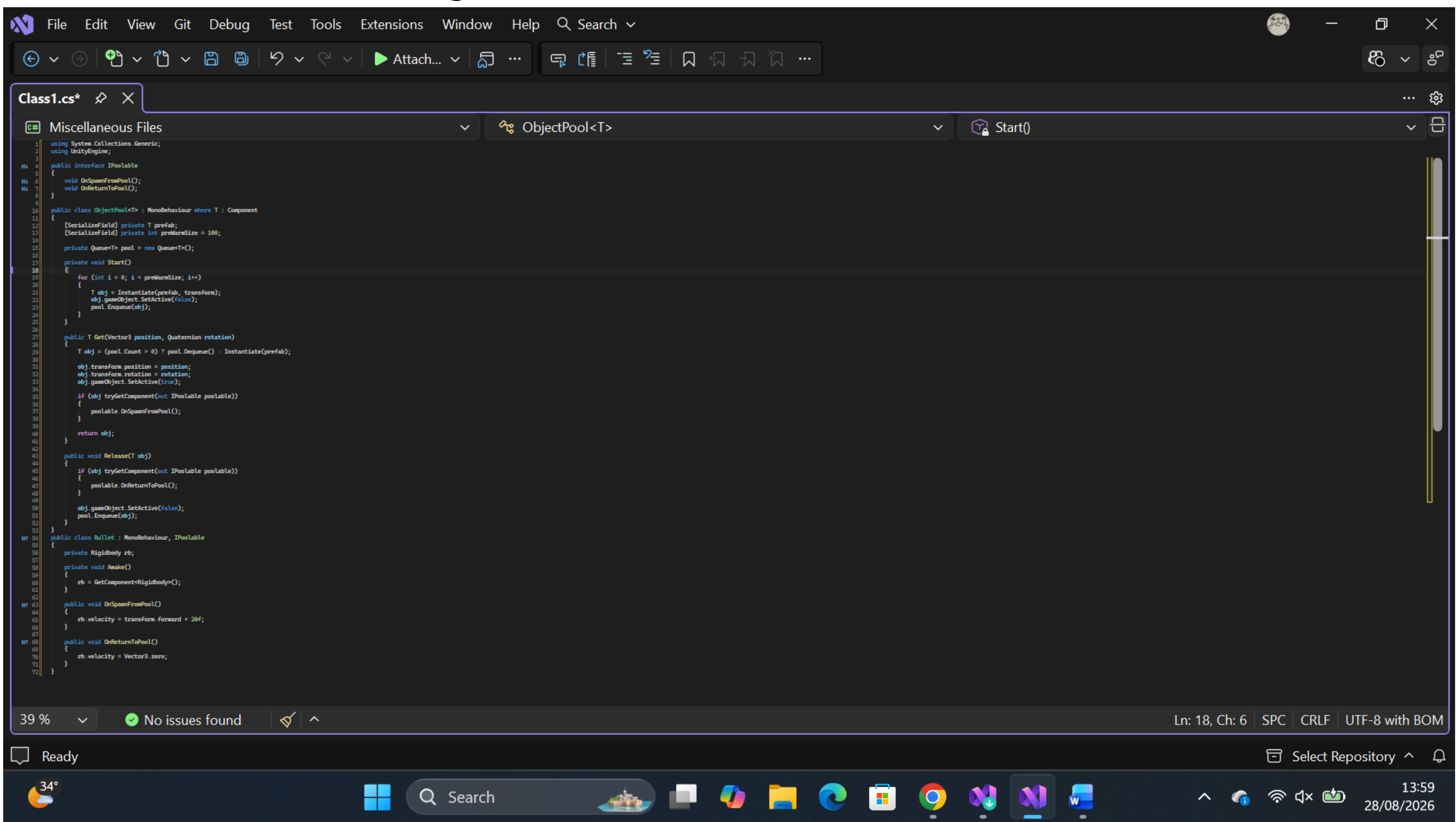
1. Mekanisme Garbage Collector (GC) & Tekanan GC

- **Mark-and-Sweep:** GC bekerja dengan memindai seluruh reference objek aktif di memori (Marking), lalu menghapus objek yang sudah tidak terpakai/dihancurkan (Sweeping).
- **Generational GC:** Membagi objek menjadi beberapa generasi (Gen 0, 1, 2) berdasarkan usianya. Objek baru (seperti peluru hasil Instantiate) masuk ke Gen 0.
- **Penyebab Stuttering:** Instantiate() dan Destroy() yang dipanggil ratusan kali per detik membuat memori terfragmentasi dan heap cepat penuh (GC Pressure). Hal ini memicu jeda stop-the-world (GC menghentikan eksekusi game sementara untuk membersihkan memori) yang terasa sebagai frame drop/stutter.

2. Analisis Amortized Complexity Object Pooling

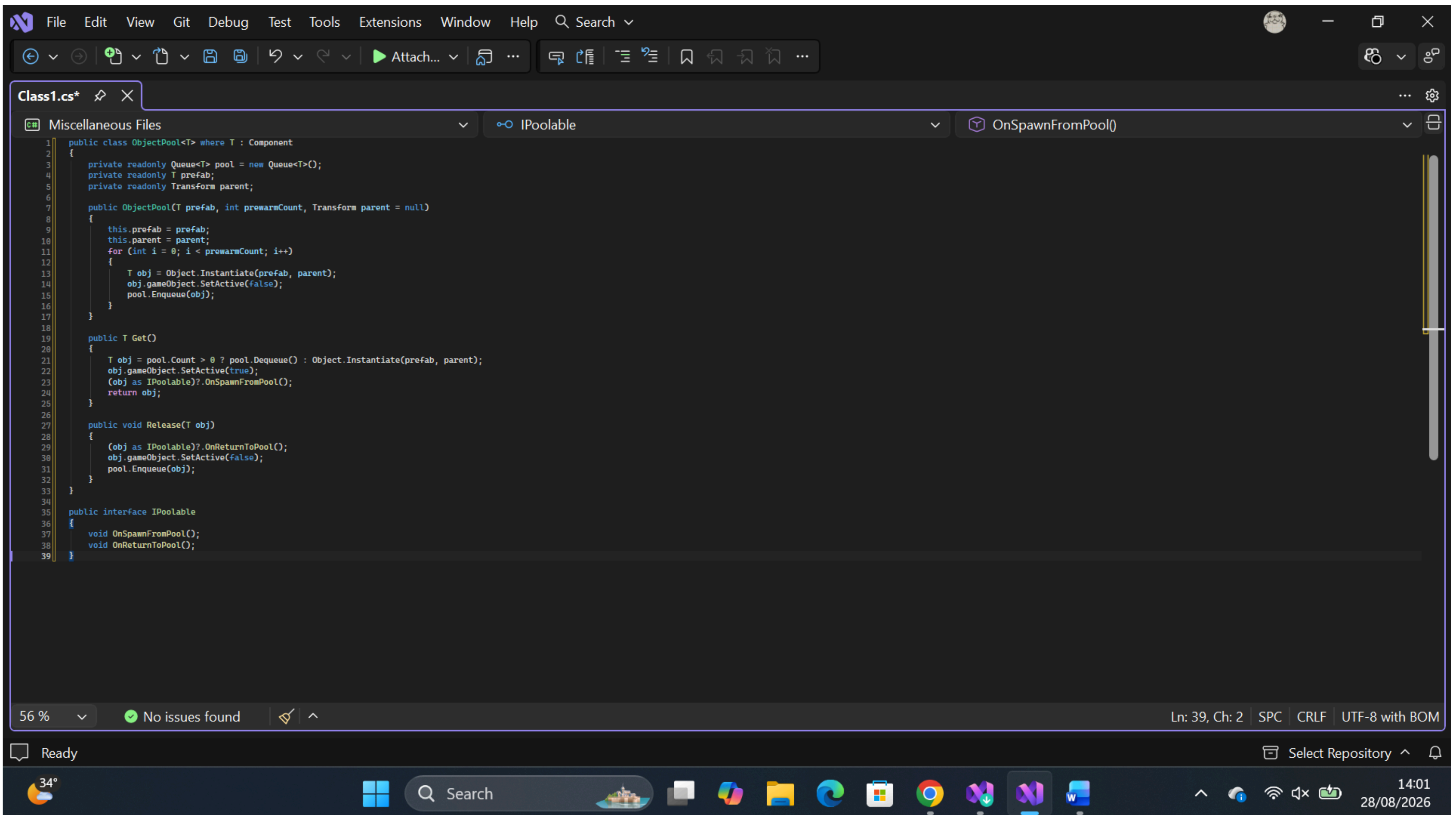
- Pendekatan Naif (`Instantiate` / `Destroy`): Biaya alokasi dan dehalokasi bernilai $O(1)$ untuk setiap peluru, sehingga total biaya untuk N peluru dalam waktu T adalah $O(N \times T)$.
- Object Pooling:
 - Biaya awal (*Pre-warming*): Membuat N objek sekaligus saat *loading/Start* memiliki biaya awal $O(N)$.
 - Kondisi *Steady-State*: Pengambilan (`Get`) dan pengembalian (`Release`) dari *queue/stack* berbiaya $O(1)$ tanpa ada alokasi/dealokasi memori baru.
 - **Amortized Cost**: Total biaya rata-rata per-peluru mendekati $O(1)$ mendekati nol karena biaya alokasi $O(N)$ disebar ke ribuan kali penggunaan ulang objek selama permainan berjalan.

3. Kode C# (ObjectPool<T> & Bullet)



4. Keterbatasan & Risiko Object Pooling

- **Stale State Bug:** Jika variabel/data objek (seperti sisa HP, speed, atau jejak visual TrailRenderer) lupa di-reset saat dimasukkan kembali ke pool, objek baru yang di-Get akan membawa status/bug lama dari pemakaian sebelumnya.
- **Pemborosan Memori (Idle Memory Cost):** Objek yang di-prewarm dalam jumlah besar (misal: 500 peluru) akan terus memakan kapasitas RAM sepanjang game berjalan, meskipun saat itu game sedang tidak memunculkan banyak peluru.



TANTANGAN 8

Esai Perbandingan: OOP Inheritance vs Entity

Component System (ECS)

Perkembangan arsitektur perangkat lunak dalam pengembangan gim melahirkan dua pendekatan utama, yaitu Object-Oriented Programming (OOP) berbasis pewarisan (inheritance) dan Entity Component System (ECS) berbasis Data-Oriented Design (DOD). Keduanya memiliki filosofi, keunggulan, serta area penggunaan yang berbeda.

1. Cache Locality & Tata Letak Memori

- **OOP (Array of Structs / AoS):** Setiap objek menyimpan seluruh datanya secara bersamaan (misal: posisi, HP, nama, animasi) di memori heap. Tata letak ini acak dan terfragmentasi. Saat CPU memperbarui posisi ribuan objek, CPU terpaksa memuat data yang tidak dibutuhkan ke cache line, yang sering memicu cache miss (kinerja CPU melambat).
- **ECS (Struct of Arrays / SoA):** Data sejenis dipisahkan dan disimpan berdampingan dalam array linier yang kontigu (misal: seluruh data posisi disimpan dalam satu array khusus). CPU prefetcher dapat memuat data ini secara berurutan dan efisien (cache locality tinggi), sehingga pemrosesan ribuan data sejenis menjadi jauh lebih cepat.

2. Komposisi vs Pewarisan (Composition over Inheritance)

- **OOP (Fragile Base Class Problem):** Pada hierarki kaku (GameCharacter -> Player/Enemy), mengubah kode pada class induk berisiko merusak perilaku class turunan secara tidak terduga. Menambahkan fitur baru yang tidak dimiliki semua turunan juga sulit dilakukan tanpa merusak struktur.
- **ECS (Komposisi Bebas):** Entitas hanyalah sebuah ID tanpa logika. Fitur atau kemampuan ditambahkan dengan menempelkan Component data yang independen. Mengubah atau menambah fitur pada satu jenis entitas tidak akan merusak entitas lain karena setiap komponen berdiri sendiri.

3. Kondisi di Mana OOP Tetap Lebih Baik

Meskipun ECS sangat cepat, OOP tetap menjadi pilihan utama pada beberapa kondisi:

- Skala Proyek Kecil-Menengah: OOP lebih cepat dirancang dan diimplementasikan tanpa overhead arsitektur yang rumit.**
- Kecepatan Pengembangan (Development Velocity): Sebagian besar pengembang sudah sangat familiar dengan OOP, sehingga alur kerja menjadi lebih lancar.**
- Kemudahan Debugging: Logika OOP mudah ditelusuri baris demi baris, sedangkan ECS memiliki alur eksekusi terpisah antara data dan logika yang membuat debugging jauh lebih kompleks.**

4. Tabel Perbandingan (Trade-Off)

OOP memiliki keunggulan pada kemudahan debugging, kecepatan pengembangan yang tinggi untuk tim umum, dan sangat cocok bagi tim pemula, meskipun memiliki performa yang rendah pada skala besar akibat seringnya terjadi cache miss. Sebaliknya, ECS menawarkan performa yang sangat tinggi pada skala besar karena tata letak memorinya yang ramah CPU cache, namun memiliki proses debugging yang sulit, kecepatan pengembangan awal yang lambat karena kurva pembelajaran yang tinggi, serta kurang cocok untuk tim pemula.

Rekomendasi Akhir: Gunakan OOP untuk gim skala kecil-menengah, sistem UI, atau proyek dengan tenggat waktu cepat karena lebih mudah diimplementasikan dan di-debug oleh tim pemula, sedangkan ECS sebaiknya dipilih ketika membangun gim berkinerja tinggi yang membutuhkan simulasi ribuan objek sekaligus di layar seperti genre Bullet-Hell, RTS, atau simulasi kerumunan massal.

**ARSYA XI PPLG 1 BLOCKING DAY 5 (jum'at 28
agustus,2026)**